Py

Day 15/55 - Operators in Python

Arithmetic, assignment, comparison, and logical operations

Prepared by Satyanarayan Sen

Video Summary

This Telusko lesson introduces Python operators used for calculations, assignments, comparisons, and logical decisions. Operators help combine values and conditions into useful program expressions.

Source video: https://www.youtube.com/watch?v=v5MR5JnKcZI

1 Operator Categories

Operators are symbols or keywords that perform work on values. This lesson groups them into arithmetic, assignment, relational, and logical operators.

+ Arithmetic

Perform math such as add, divide, quotient, and remainder.

= Assignment

Store or update values in variables.

== Relational

Compare values and return True or False.

operator overview

```
>>> 5 + 3
8
>>> x = 5
>>> x += 2
>>> x
7
>>> 7 > 3 and 2 < 5
True
```

2 Arithmetic Operators

Arithmetic operators perform mathematical work on numbers. Python also supports floor division for quotients and modulus for remainders.

Operator	Meaning	Example
`+`	Addition.	`5 + 3 -> 8`
`-`	Subtraction.	`5 - 3 -> 2`
`*`	Multiplication.	`5 * 3 -> 15`
`/`	Division returns decimal result.	`5 / 2 -> 2.5`
`//`	`Floor division returns quotient.`	`5 // 2 -> 2`
`%`	Modulus returns remainder.	`5 % 2 -> 1`

arithmetic examples

```
>>> a, b = 10, 3
>>> a / b
3.3333333333333335
>>> a // b, a % b
(3, 1)
>>> -a
-10
```

Unary operator

A unary operator works on one value. For example, if x = 5, then -x gives -5.

3 Assignment Operators

Assignment operators store values in variables. Shorthand forms update an existing value without repeating the full expression.

Form	Meaning	Example result
`x = 5`	Assign 5 to x.	x becomes 5
`x += 2`	`Same as x = x + 2.`	5 becomes 7
`x -= 2`	`Same as x = x - 2.`	7 becomes 5
`x *= 3`	`Same as x = x * 3.`	5 becomes 15
`a, b = 5, 6`	Assign multiple variables in one line.	a is 5, b is 6

assignment examples

```
>>> x = 5
>>> x += 3
>>> x
8
>>> a, b = 10, 20
>>> a, b
(10, 20)
```

Shorthand rule

Use +=, -=, *=, and similar forms when you want to update a variable based on its current value.

4 Relational Operators

Relational operators compare values. The result is always a Boolean value: True or False.

> **Greater than**

Checks whether the left value is larger.

== **Equal to**

Checks whether two values are equal.

!= **Not equal**

Checks whether two values are different.

Operator	Meaning	Example
`>`	Greater than.	`7 > 5 -> True`
`<`	Less than.	`7 < 5 -> False`
`>=`	`Greater than or equal to.`	`7 >= 7 -> True`
`<=`	`Less than or equal to.`	`5 <= 7 -> True`
`==`	Equal to.	`5 == 5 -> True`
`!=`	Not equal to.	`5 != 7 -> True`

comparison examples

```
>>> a, b = 7, 5
>>> a > b, a == b, a != b
(True, False, True)
>>> a < b
False
```

5 Logical Operators

Logical operators combine or reverse conditions. They are essential for control flow because they decide how multiple Boolean conditions work together.

Operator	Meaning	True when
`and`	Both conditions must be true.	`True and True`
`or`	At least one condition must be true.	`True or False`
`not`	Reverses the Boolean value.	`not False`

A	B	A and B	A or B
False	False	False	False
False	True	False	True
True	False	False	True
True	True	True	True

logical examples

```
>>> c1 = True; c2 = False
>>> c1 and c2
False
>>> c1 or c2
True
>>> not c1
False
```

6 Practice Code

Run this small script to practice arithmetic, assignment, relational, and logical operators together.

operators practice

```
a = 10
b = 3

print(a + b, a - b, a * b)
print(a / b, a // b, a % b)

x = 5
x += 4
print(x)

print(a > b, a == b, a != b)
print(a > 5 and b < 5)
print(a < 5 or b < 5)
print(not (a == b))
```

Idea	What to watch	Meaning
`a // b`	**`Floor division.`**	Quotient without decimal.
`a % b`	Modulus.	Remainder after division.
`and / or / not`	Logical combination.	Controls condition results.

Day 15/55 takeaway

Operators let Python calculate, assign, compare, and combine conditions. Master arithmetic, assignment, relational, and logical operators before writing larger decision-based programs.